ZEE COLLECTION: Consolidated Phase 0 Architecture Blueprint, Decision Register, and Pre-Implementation Governance Report

Executive Governance Summary and Implementation Gate Status

The **ZEE COLLECTION** (zeecollection.tech) e-commerce initiative remains strictly in **Phase 0 (Discovery, Requirements Reconciliation, and Architecture Blueprinting)**. In accordance with the project's governing framework and explicit directives, **implementation is paused and not authorized**. No project files have been created, no frontend or backend source code has been written or modified, no Cloudflare resources (Pages projects, Pages Functions, D1 databases, or R2 storage buckets) have been provisioned, no DNS records or nameserver configurations have been altered, and no deployments have been executed.
This architectural report synthesizes the foundational governance rules from the *Zero-to-Production Master Project Prompt* with the later project decisions recorded in the *Phase 0 Handover and Decision Register*, while integrating four authoritative project corrections. First, **watches** constitute the sole approved initial product catalog, while **rings and glasses** are designated for potential future expansion, superseding all legacy references to footwear and European shoe sizing. Second, **three independent administrator accounts** must operate with **equal permissions** across the entire administrative surface, encompassing manual payment verification, refund management, administrator account management, security configuration, and protected audit-log access, thereby replacing earlier single-administrator assumptions. Third, **customer notifications** must support **website notifications** and **email notifications** (subject to technical verification of the outbound mail path), while **WhatsApp** (+923408133043) is restricted strictly to **click-to-chat customer support** rather than automated transactional messaging or order persistence. Fourth, execution remains frozen behind the mandatory phase-completion gate protocol until the project owner issues the exact authorization phrase START PROJECT.

Reconciliation of Material Conflicts and Newly Confirmed Decisions

Cross-referencing the original Master Prompt against the Phase 0 Handover and the latest confirmed directives exposes three material historical conflicts alongside functional gaps in the unrecovered portions of the 24-question discovery register. Reconciling these conflicts prior to Phase 1 prevents structural flaws in the relational database schema, administrative access control logic, and third-party integration boundaries.

Architectural Domain	Legacy Master Prompt / Earlier Handover State	Confirmed Override & Corrected Baseline	Architectural & Database Impact
Primary Product Catalog & Variants	Contradictory records citing "Premium shoes and fashion e-commerce" (Sneakers, Formal Shoes, Casual Shoes; sizes EU 40–45) alongside a conflicting note mentioning watches, rings, and glasses.	**Watches** are the sole approved initial product catalog. **Rings** and **glasses** may be introduced in later phases. All footwear categories and EU shoe-sizing schemas are retired.	The products and product_variants tables in D1 must model horological attributes (strap type, dial color, case size, model) while utilizing extensible variant keys (option_type, option_value) to accommodate ring sizes and eyewear frames later without destructive schema migrations.
Administrative Identity & Permissions	Recorded a single preferred built-in login identifier (admin@zeecollection.tech) without finalized multi-user role definitions.	**Three independent administrator accounts** with **equal, full permissions** covering catalog CRUD, payment verification, refunds, account management, security settings, and audit-log access.	Eliminates shared-credential usage; requires individual admin_id attribution on every state change, optimistic concurrency locking in D1 to prevent conflicting simultaneous actions by peer admins, and anti-lockout safeguards in account management.
Customer Notifications & WhatsApp Scope	Ambiguity regarding whether WhatsApp (+923408133043) would handle order submission or automated alerts; unverified Zoho email (info@zeecollection.tech) recorded.	**Website notifications** and **email** (subject to verification) handle customer notifications. **WhatsApp** is strictly **click-to-chat support**, not automated messaging.	Requires an on-site notification persistence table (customer_notifications) tied to secure order tracking as the guaranteed primary channel, an abstracted email dispatcher pending Zoho/HTTP-email verification, and static/dynamic wa.me deep links for support.
Implementation &	Handover transferred at	**Implementation**	Enforces strict

Architectural Domain	Legacy Master Prompt / Earlier Handover State	Confirmed Override & Corrected Baseline	Architectural & Database Impact
Resource State	Phase 0 boundary with partial discovery gaps (Q9–Q12, Q16–Q18, Q23–Q24).	**remains paused.** No code, files, Cloudflare bindings, or DNS changes are permitted before START PROJECT.	pre-implementation blueprint consolidation and isolates remaining unresolved operational parameters before Phase 1 begins.

Architectural Implications of the Catalog Shift to Watches

Transitioning the initial catalog from footwear to watches fundamentally alters how inventory variants and visual assets behave in an e-commerce backend. Footwear catalogs typically require a uniform, multi-dimensional matrix where nearly every product is split across six or more standardized sizes (such as EU 40–45) and multiple colors, resulting in high row multiplication in the variants table. Watches, by contrast, frequently sell either as standalone single-SKU models (where the timepiece itself is unique) or with single-dimension variations such as strap material (leather, stainless steel, mesh), case finish (gold, silver, matte black), or dial color.

This operational profile aligns directly with the confirmed decision in **Q5 (Option B)**, which stipulates that variants are optional per product, products without variants must be supported natively, and advanced multi-axis variant combinations are not required at launch. Furthermore, because **rings** (which require discrete ring-size variants) and **glasses** (which require lens tint or frame-width variants) may be added in subsequent phases, the Cloudflare D1 schema must avoid hardcoded attribute columns such as shoe_size or watch_strap. Defining a generic option_type (such as "Strap / Finish" or "Case Diameter" at launch, and "Ring Size" or "Frame Style" in the future) paired with option_value per variant row ensures that expanding the store into rings and glasses requires zero destructive database migrations. Additionally, horological photography relies heavily on macro detail—capturing dial texture, bezel finish, and clasp craftsmanship—against the preferred luxury/minimalist **black, white, and gold** visual palette, reinforcing the requirement for high-clarity WebP optimization in Cloudflare R2 across the target of three photos per product.

Distributed Concurrency and Security Implications of Three Equal-Permission Administrators

Mandating three independent administrator accounts with identical permissions across payment verification, refunds, account management, security settings, and audit-log access introduces three critical distributed-systems and security requirements that do not exist in a single-administrator store. Because all three administrators operate from mobile browsers and share equal authority to verify payment evidence, approve or reject customer cancellation requests (**Q19**), and record refunds (**Q7**), two administrators could inspect and act upon the same pending order simultaneously. Without row-level concurrency guards, one administrator could approve a customer's cancellation request (releasing reserved watch stock back to sellable inventory) at the exact moment a second administrator marks the uploaded payment receipt as verified and transitions fulfillment to processing. To prevent race conditions and double-restocking, every administrative mutation in Cloudflare Pages Functions must pass the

expected state_version integer and execute conditional atomic SQL updates (UPDATE orders SET ..., state_version = state_version + 1 WHERE id = ? AND state_version = ?) inside a D1 batch transaction. If zero rows are affected by the conditional update, the backend immediately rejects the concurrent request with an HTTP 409 Conflict status and prompts the administrator's mobile interface to refresh the order state.
Equally important is cryptographic attribution and append-only audit integrity. When three administrators share identical privileges—including security settings and account management—a single shared login (admin@zeecollection.tech) cannot satisfy non-repudiation requirements. Each of the three administrators must authenticate with a distinct email identifier and an independent password hash, generating a server-side session bound to a unique admin_id. While all three administrators hold full read access to the protected audit logs, the backend API and D1 data access layer must enforce strict append-only semantics (INSERT only, with zero UPDATE, DELETE, or TRUNCATE endpoints) on the audit_logs table so that no administrator can erase evidence of a payment verification, refund approval, or security configuration change.
Furthermore, granting equal account-management and security-settings permissions to all three administrators creates a peer-lockout hazard where one compromised account—or an accidental mobile interface tap—deactivates or deletes the other two administrator accounts, or locks out all three accounts simultaneously. Consequently, the administrative backend must enforce invariant self-preservation guards: the database must maintain the three administrator account slots with a hard rule that at least two administrator accounts remain active (is_active = 1) at all times, require step-up password re-verification from the acting administrator before any credential or security-setting modification is committed, immediately log all account-management events to audit_logs, and revoke active rows in admin_sessions whenever an account's password or security status is rotated.

Operational Implications of the Notification and Email Architecture

Establishing **website notifications** and **email** (subject to verification) as the customer notification channels while restricting **WhatsApp** (+923408133043) to **click-to-chat support** decouples core order reliability from external messaging APIs. Automated WhatsApp Business API integrations require Meta business verification and incur per-conversation fees through third-party providers, which would violate the strict **free-tier budget constraint**. Implementing WhatsApp strictly as a client-side https://wa.me/923408133043?text=... click-to-chat trigger—which pre-fills the customer's public Order Reference ID when tapped from an order tracking screen—delivers immediate mobile support convenience at zero infrastructure cost and zero server-side failure risk.
For email notifications, external technical verification highlights a critical platform constraint involving the previously recorded **Zoho Mail** setup (info@zeecollection.tech) and **Cloudflare Pages Functions**. Zoho Mail's Forever Free plan supports up to 5 users (with 5 GB storage each) on a single custom domain at zero cost, which is sufficient to host info@zeecollection.tech alongside three individual administrator mailboxes. However, Zoho Mail Free restricts external POP, IMAP, and SMTP protocol access to webmail and official Zoho mobile apps on standard free accounts, and even where outbound SMTP is enabled, Zoho enforces conservative dynamic sending limits of approximately **50 outgoing emails per day and 10 emails per hour** for free users. Simultaneously, Cloudflare Pages Functions execute on Cloudflare's edge V8 isolate runtime, which is designed around asynchronous HTTP fetch() calls to transactional email APIs rather than legacy TCP SMTP socket handshakes. Therefore, **website notifications**

persisted in Cloudflare D1 and displayed directly inside the customer's Order Tracking & Notification Center on zeecollection.tech must serve as the **authoritative, guaranteed primary notification channel** from day one. Outbound automated email notifications must be encapsulated behind a modular, non-blocking notification adapter that remains toggled off in Admin Settings until the outbound email delivery path (and Cloudflare DNS MX, SPF, DKIM, and DMARC records) is verified to operate reliably within free-tier boundaries. Under no circumstances may an email rate-limit rejection or delivery timeout roll back or block an order placement or administrative status update.

Consolidated Phase 0 Decision Register (All 24 Discovery Domains)

The Phase 0 Handover preserved explicit decisions for 14 of the 24 numbered discovery questions (Q3–Q8, Q13–Q15, Q19–Q22), partial reconstructed preferences for 3 questions (Q1, Q2, Q23), and noted that verbatim question wordings and answers for 7 questions (Q9–Q12, Q16–Q18, Q24) were unavailable in the exported record. In strict compliance with anti-hallucination rules, the missing verbatim text for those 7 questions is never fabricated. Cross-referencing the Master Prompt's mandatory discovery domains (Section 6.A through 6.G) alongside the project owner's four confirmed overrides demonstrates that key functional areas originally covered by those unrecovered questions—specifically multi-administrator permissions, audit logging, customer notifications, and WhatsApp scope—are now definitively resolved, while platform capacity remains bounded by Cloudflare and Zoho free-tier quotas.
The complete status of all 24 Phase 0 questions and their reconciliation with the four latest project confirmations is detailed in the register below:

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
Q1	Business identity, target market, currency, and catalog	**Confirmed & Corrected** (Partial original wording; updated via Confirmation #1)	Store: **ZEE COLLECTION** (zeecollection.tech). Market: **Pakistan**. Language: **English**. Currency: **PKR** (displayed as whole rupees). Visual style: **Luxury/minimalist** (black, white, gold). Initial catalog: **Watches** (15 initial target, ~50 max planned, 3 photos/product). **Rings and**	Confirm initial watch category subdivisions (e.g., Men's, Women's, Luxury, Casual, Chronograph) and whether any legal entity details appear in the footer.

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
			glasses may be considered later.	
Q2	Hosting, development tools, and infrastructure	**Confirmed** (Reconstructed baseline; external state unverified)	Primary environment: **Android phone + SPCK Editor**. Source control: **GitLab** (clean start from zero). Infrastructure: **Cloudflare Free Tier** (Pages, Pages Functions, D1, R2). No paid services without explicit prior approval.	Inspect and verify current domain registrar, nameservers, Cloudflare account status (including R2 activation requirements), and GitLab repo setup in Phase 1.
Q3	Payment-provider strategy	**Confirmed** (Option E)	Payment providers remain undecided at the code level; architecture must provide **admin-configurable Payment Settings, Shipping Settings, and Order Management** so methods can be toggled without code rewrites.	Verify actual merchant eligibility and operational terms before enabling any payment method in production.
Q4	Product catalog and inventory management	**Confirmed** (Option B)	Administrators manage catalog via protected backend. **D1** stores authoritative product/inventory records; **R2** stores product images. Server validates all prices and stock; concurrent orders must never	None at design level; requires implementation and concurrency testing in Phases 3 and 9.

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
			oversell.	
Q5	Product variants	**Confirmed & Adapted** (Option B + Confirmation #1)	Variants are optional per watch (e.g., strap material/color, dial color, case size). Each variant has its own **SKU, whole-PKR price, exact stock quantity, and active status**. Single-SKU watches without variants are also supported. Order snapshots are immutable.	Confirm the default variant labels needed for the initial 15 watches.
Q6	Stock reservation and expiry	**Confirmed**	Atomically reserve stock at order placement using concurrency-safe D1 transactions. Unpaid reservations **expire after 24 hours**. Cancellations or expiries release stock only under validated transition rules.	Exact hybrid lazy-evaluation and administrative sweep transition logic is defined in the Technical Blueprint below.
Q7	Returns, exchanges, and refunds	**Confirmed** (Option A)	Workflow must be documented before launch. Admin reviews and manages returns; returned watches must be inspected and approved before restocking. **Refund approval is not proof of funds returned.**	Project owner must confirm specific customer-facing return window (e.g., days after delivery) and exchange eligibility rules aligned with Pakistani law.

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
			Full history preserved.	
Q8	Customer contact, support, and notifications	**Confirmed & Corrected** (Option C + Confirmation #3)	Channels: Website contact form (server-validated, spam-protected, persisted in D1), **Email** (info@zeecollection.tech, Zoho recorded), and **WhatsApp** (+923408133043). Customer notifications use **website notifications and email (subject to verification)**. WhatsApp is **click-to-chat support only**, not automated messaging.	Verify live Zoho mailbox status, DNS MX/SPF/DKIM records, outbound email capability from Cloudflare, and WhatsApp number responsiveness.
Q9	*Original wording & final decision unavailable*	**Unrecovered Verbatim** (Partially addressed by Confirmations #2 & #3)	Exact historical text is not in the recoverable record and is not invented. In the Master Prompt discovery structure (Section 6.D/6.E), adjacent items covered checkout/account mode, admin roles, and notifications.	See consolidated unresolved table below for guest vs. account checkout confirmation.
Q10	*Original wording & final decision unavailable*	**Unrecovered Verbatim**	Exact historical text unavailable; no decision inferred.	Covered under consolidated unresolved check below.
Q11	*Original wording & final decision*	**Unrecovered Verbatim**	Exact historical text unavailable;	Covered under consolidated

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
	unavailable		no decision inferred.	unresolved check below.
Q12	*Original wording & final decision unavailable*	**Unrecovered Verbatim**	Exact historical text unavailable; no decision inferred.	Covered under consolidated unresolved check below.
Q13	Backups and recovery	**Confirmed** (Option A)	Critical D1 records and R2 objects require a **documented and tested backup and recovery plan** within zero-cost constraints (leveraging D1 Time Travel + SQL/JSON admin export and R2 archive download to mobile/GitLab private backup).	Establish exact backup frequency, retention period, RPO, and RTO targets prior to launch.
Q14	Shipping and courier integration	**Confirmed** (Option C)	**Manual, administrator-managed shipping at launch**; modular architecture to allow courier API integration later.	Confirm exact dispatch locality around **Kotla / Gujrat, Punjab**, and verify courier (e.g., Leopards or alternative) pickup, coverage, COD settlement, and fee schedule.
Q15	Product image uploads and storage	**Confirmed** (Option A)	Secure admin uploads persisted in **R2** with metadata/references in **D1**. Server validates file signatures and sizes; images optimized into responsive sizes. **Private payment evidence** stored	Verify exact max upload size, accepted MIME types, and client-side HTML5 Canvas WebP optimization behavior on Android.

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
			and accessed strictly separately from public watch images.	
Q16	*Original wording & final decision unavailable*	**Unrecovered Verbatim** (Admin domain resolved by Confirmation #2)	Exact historical text unavailable. However, the project owner's latest directive explicitly resolves the administrative model: **three independent administrator accounts with equal permissions** (payment verification, refunds, account management, security settings, and audit-log access).	Confirm the three specific administrator email identifiers to be seeded or registered during Phase 7 setup.
Q17	*Original wording & final decision unavailable*	**Unrecovered Verbatim**	Exact historical text unavailable; no decision inferred.	Covered under consolidated unresolved check below.
Q18	*Original wording & final decision unavailable*	**Unrecovered Verbatim**	Exact historical text unavailable; no decision inferred.	Covered under consolidated unresolved check below.
Q19	Customer cancellation requests	**Confirmed** (Option A)	**Customers request cancellation; administrators decide.** Request alone never cancels an order, releases stock, or triggers a refund. Admin reviews order/payment/fulfillment state,	Confirm customer-facing cancellation request policy terms (e.g., whether requests are blocked once Shipping is marked Shipped).

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
			approves or rejects, and logs the decision in protected audit records.	
Q20	Order, payment, fulfillment, and shipping statuses	**Confirmed** (Option A)	**Four separate status tracking domains** (Order, Payment, Fulfillment, Shipping) with validated server-side transitions, audit logging, and sanitized customer-facing views.	Finalized state transition rules and cross-domain invariants are specified in the Technical Blueprint below.
Q21	Launch blockers and verification	**Confirmed** (Option A)	**Documented verification evidence required** in TESTING.md across all 18 launch blockers before the store can be considered production-ready.	Executed progressively across Phases 1–9.
Q22	Payment methods and activation	**Confirmed** (Option C)	**Only verified payment methods may be enabled** (candidates: COD, Bank Transfer, Easypaisa/JazzCash, Raast, or verified gateway). No unverified or paid provider without explicit approval.	Verify bank/mobile-wallet account readiness and COD courier settlement terms before enabling at checkout.
Q23	Shipping coverage and charges	**Partially Reconstructed** (Final decision	Earlier preference noted a **PKR 250** delivery charge	Confirm whether PKR 250 is a flat nationwide fee or

Question ID	Decision Domain	Recorded Status	Consolidated & Corrected Specification	Remaining Verification / Gap
		unconfirmed)	separate from product price, but this is not yet confirmed as a universal production rule.	varies by region/weight/COD vs. prepaid, and whether a free-shipping threshold applies.
Q24	*Original wording & final decision unavailable*	**Unrecovered Verbatim**	Exact historical text unavailable; no decision inferred.	Covered under consolidated unresolved check below.

Analytical Mapping of the Unrecovered Discovery Items (Q9–Q12, Q16–Q18, Q24)

Rather than restarting discovery from zero or fabricating historical text, a systematic comparison between the Master Prompt's discovery checklist (Section 6.A through 6.G) and the 17 known questions in the Handover reveals the exact functional topics that resided inside the unrecovered question slots. Specifically, the multi-administrator structure, staff role equality, security settings access, and audit-log requirements (Section 6.E of the Master Prompt) are now **fully resolved** by Confirmation #2, which establishes three independent administrators with equal permissions. Similarly, order confirmation channels, customer notification methods, and the exact operational boundary of WhatsApp (Section 6.D of the Master Prompt) are now **fully resolved** by Confirmation #3, which mandates website notifications and verified email while restricting WhatsApp strictly to click-to-chat customer support.

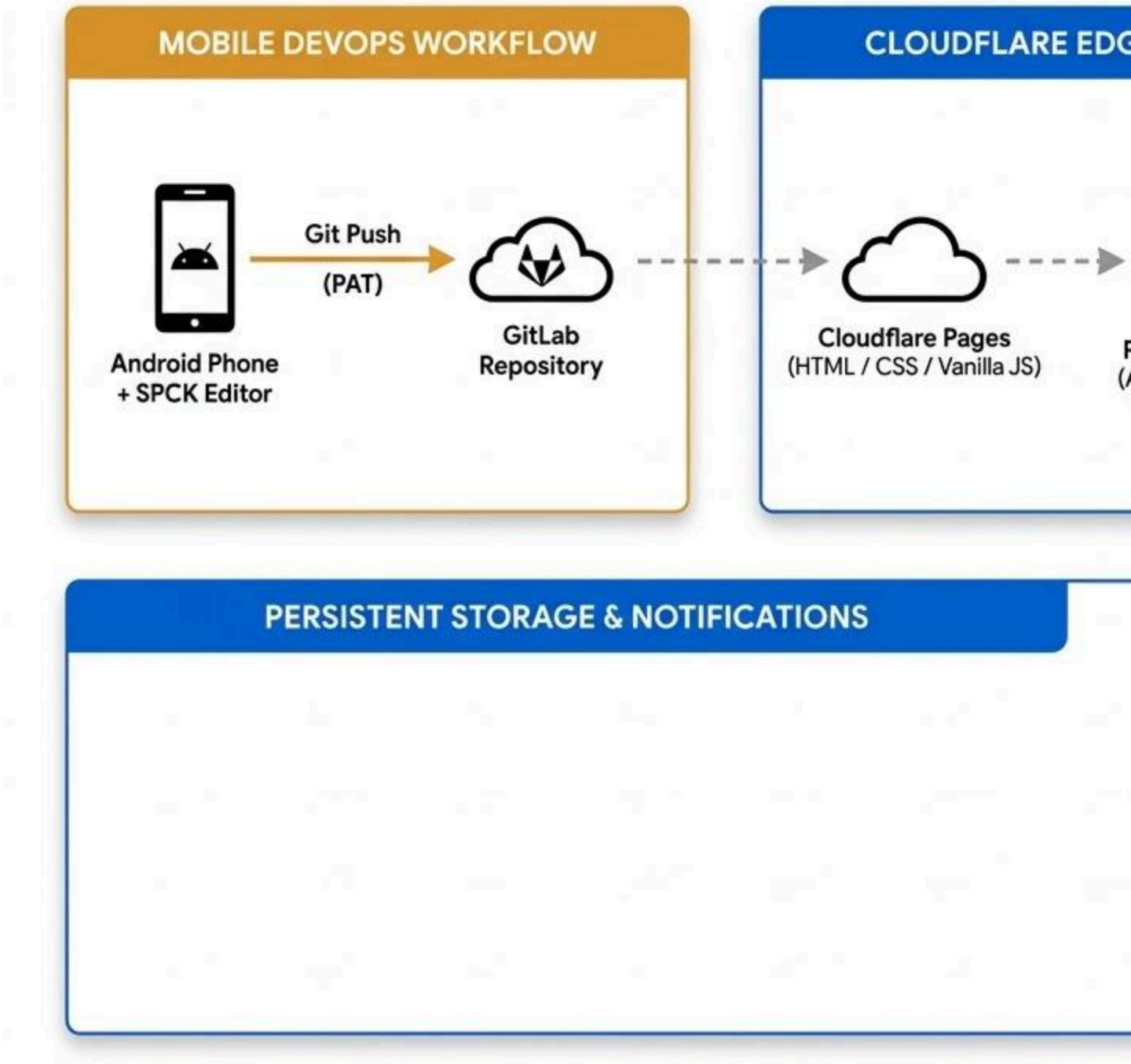
Comparing the remaining Master Prompt discovery items against the Handover leaves only a narrow set of practical product and workflow parameters that would have resided in Q9–Q12, Q17–Q18, or Q24. First, **Customer Checkout and Order Tracking Access Mode**—referenced in Q19 and Q20 as *"through secure order tracking or an authenticated account, where applicable"*—requires confirming whether customers check out as guests and access their order status and website notifications via a secure Order Reference and cryptographic Tracking Token/PIN, or whether customer account registration is required at launch. Second, **Storefront Merchandising Features** (Master Prompt Section 2 and Section 6.C) involve confirming whether product search, category/price filtering, sale price badges, and scheduled product publishing are all desired in the initial release alongside standard Published, Draft, and Archived statuses. Third, operational parameters for the **Backup Schedule and Recovery Objectives** (Q13 gap) and the **Final Shipping Rate and Return Window Rules** (Q7 and Q23 gaps) remain isolated for confirmation prior to launch.

Corrected Technical Blueprint (BLUEPRINT.md Baseline Specification)

System Architecture and Mobile-First Development Pipeline

The ZEE COLLECTION platform is architected as a zero-cost, serverless edge application

running natively on the Cloudflare developer platform and maintained entirely from an **Android smartphone** using **SPCK Editor**, **GitLab Web UI**, and the **Cloudflare Mobile Dashboard**.



Every operational step in this architecture is engineered to execute without a desktop computer, local command-line terminal, or Node.js build pipeline. At the presentation layer, the storefront

and administrative interface utilize modular, standards-compliant **HTML5, CSS3, and Vanilla ES6+ JavaScript**. By avoiding heavy frontend compilation frameworks that require npm run build or local bundlers, files edited in SPCK Editor on Android are immediately deployable as static assets while communicating with backend endpoints via asynchronous fetch() requests with JSON payloads and multipart FormData streams for binary uploads.
Source control synchronization between SPCK Editor on Android and GitLab relies on SPCK Editor's built-in isomorphic-git engine, which natively supports cloning repositories, staging files, committing changes, switching branches, resolving merge diffs, and pushing commits to remote origins. Authentication with GitLab is performed using a **GitLab Personal Access Token (PAT)** or Project Access Token configured with api (or read_repository and write_repository) permissions, stored in SPCK Editor under Git Menu -> Git Credentials -> GitLab. Because SPCK Editor stores working project files inside internal Android application storage that is wiped if the app is uninstalled, the operational workflow requires committing and pushing changes to GitLab after every logical edit and periodically exporting a local .zip project archive from SPCK Editor to Android device storage.
From GitLab, continuous deployment is handled through **Cloudflare Pages' native GitLab integration** (Workers & Pages -> Create application -> Pages -> Connect to Git -> GitLab), which supports both private and public repositories provided the GitLab account holds the Maintainer or Owner role and at least one branch (main) has been pushed. Every commit pushed to main automatically deploys production static files and compiles the /functions directory into **Cloudflare Pages Functions** across Cloudflare's global network without requiring the desktop wrangler CLI. Whenever documentation-only edits are pushed that do not require rebuilding the live site, prefixing the commit message with [CF-Pages-Skip] or [CI Skip] instructs Cloudflare Pages to omit the build run. Meanwhile, Cloudflare D1 schema creation, SQL migrations, and table inspection are fully operable via the Cloudflare Dashboard (Storage & Databases -> D1 SQL Database -> Console / Explore Data) in an Android mobile browser, supplemented by a protected Administrative Database Migration & Health endpoint within the Pages Functions backend.

Authoritative Data Model and Cloudflare D1 Schema Design

In compliance with mandatory data persistence rules, no authoritative catalog, inventory, order, or administrative data may ever reside in localStorage, sessionStorage, hardcoded JavaScript arrays, or static products.json files. Only the customer's unsubmitted shopping cart may utilize localStorage for temporary browser convenience, and every item in the cart is re-validated against live D1 prices and stock quantities on the server at checkout.
The Cloudflare D1 (SQLite-compatible) relational schema is structured across 15 normalized tables that accommodate the **Watches** initial catalog (and future **Rings** and **Glasses**), the **three equal-permission administrators**, the **24-hour atomic stock reservations**, the **four decoupled order status domains**, and **website plus email notifications**:

Table Name	Primary Key & Indexes	Key Columns & Data Types	Architectural Purpose & Invariant Rules
admins	id (TEXT UUID PK) Unique: email	email (TEXT), display_name (TEXT), password_hash (TEXT), password_salt (TEXT), role (TEXT	Stores the **3 independent administrator accounts** with equal full permissions.

Table Name	Primary Key & Indexes	Key Columns & Data Types	Architectural Purpose & Invariant Rules
		DEFAULT 'EQUAL_ADMIN'), is_active (INTEGER), failed_login_attempts (INTEGER), locked_until (TEXT), state_version (INTEGER), created_at (TEXT), updated_at (TEXT)	Enforces account-level brute-force lockout and optimistic locking (state_version).
admin_sessions	session_id_hash (TEXT PK) Index: admin_id, expires_at	admin_id (TEXT FK referencing admins.id), csrf_token (TEXT), ip_address (TEXT), user_agent (TEXT), created_at (TEXT), expires_at (TEXT), revoked_at (TEXT)	Server-side session store backing HttpOnly; Secure; SameSite=Strict cookies. Enables instant session revocation upon password rotation or security-setting changes.
audit_logs	id (INTEGER PK AUTOINCREMENT) Index: actor_admin_id, entity_type, created_at	actor_admin_id (TEXT FK referencing admins.id), action_type (TEXT), entity_type (TEXT), entity_id (TEXT), previous_state_json (TEXT), new_state_json (TEXT), reason (TEXT), ip_address (TEXT), created_at (TEXT)	**Append-only immutable audit ledger.** Accessible to all 3 admins for viewing; backend exposes zero update/delete operations. Logs all logins, catalog edits, payment verifications, cancellations, refunds, and security changes.
categories	id (TEXT UUID PK) Unique: slug	name (TEXT), slug (TEXT), product_line (TEXT DEFAULT 'WATCHES'), description (TEXT), sort_order (INTEGER), is_active (INTEGER), created_at (TEXT)	Supports watch collections initially (e.g., Chronograph, Dress, Minimalist, Automatic) while product_line allows seamless expansion to 'RINGS' and 'GLASSES' later.
products	id (TEXT UUID PK) Unique: slug, base_sku Index: category_id, status	category_id (TEXT FK), name (TEXT), slug (TEXT), base_sku (TEXT), description	Authoritative watch catalog record. Prices stored as **whole integer PKR**

Table Name	Primary Key & Indexes	Key Columns & Data Types	Architectural Purpose & Invariant Rules
		(TEXT), specs_json (TEXT), regular_price_pkr (INTEGER), sale_price_pkr (INTEGER NULL), has_variants (INTEGER), total_stock (INTEGER), reserved_stock (INTEGER), status (TEXT: 'DRAFT', 'PUBLISHED', 'ARCHIVED'), state_version (INTEGER), created_at (TEXT), updated_at (TEXT)	(eliminating floating-point rounding errors). For non-variant watches (has_variants = 0), sellable stock is total_stock - reserved_stock.
product_variants	id (TEXT UUID PK) Unique: sku Index: product_id	product_id (TEXT FK referencing products.id), sku (TEXT), option_type (TEXT, e.g., 'Strap / Dial'), option_value (TEXT, e.g., 'Black Leather / Gold Case'), regular_price_pkr (INTEGER), sale_price_pkr (INTEGER NULL), total_stock (INTEGER), reserved_stock (INTEGER), is_active (INTEGER), state_version (INTEGER), created_at (TEXT)	Implements **Q5 (Option B)**: optional per product, individual SKU, whole-PKR price, exact stock (total_stock - reserved_stock >= 0), and active flag. Extensible to ring sizes and eyewear later.
product_images	id (TEXT UUID PK) Index: product_id, sort_order	product_id (TEXT FK referencing products.id), r2_object_key (TEXT UNIQUE), alt_text (TEXT), width (INTEGER), height (INTEGER), size_bytes	Maps D1 watch records to persistent public **R2** image objects (supporting the initial target of 3+ photos per watch).

Table Name	Primary Key & Indexes	Key Columns & Data Types	Architectural Purpose & Invariant Rules
		(INTEGER), mime_type (TEXT), is_primary (INTEGER), sort_order (INTEGER), created_at (TEXT)	
orders	id (TEXT UUID PK) Unique: order_number Index: customer_email, order_status, created_at	order_number (TEXT), tracking_token_hash (TEXT), customer_name (TEXT), customer_email (TEXT), customer_phone (TEXT), shipping_address_json (TEXT), subtotal_pkr (INTEGER), shipping_fee_pkr (INTEGER), total_pkr (INTEGER), payment_method (TEXT), order_status (TEXT), payment_status (TEXT), fulfillment_status (TEXT), shipping_status (TEXT), courier_name (TEXT NULL), tracking_number (TEXT NULL), reservation_expires_at (TEXT), state_version (INTEGER), created_at (TEXT), updated_at (TEXT)	Enforces **Q20 (Option A)** by keeping order_status, payment_status, fulfillment_status, and shipping_status strictly separate. Uses state_version for 3-admin optimistic concurrency control.
order_items	id (TEXT UUID PK) Index: order_id	order_id (TEXT FK referencing orders.id), product_id (TEXT), variant_id (TEXT NULL), snapshot_product_name (TEXT), snapshot_variant_label	**Immutable historical purchase snapshot (Q5 & Section 4.3).** Even if an admin later edits a watch's price, name, or variant in products, historical orders remain

Table Name	Primary Key & Indexes	Key Columns & Data Types	Architectural Purpose & Invariant Rules
		(TEXT NULL), snapshot_sku (TEXT), snapshot_unit_price_pkr (INTEGER), quantity (INTEGER), line_total_pkr (INTEGER)	untouched.
stock_reservations	id (TEXT UUID PK) Index: order_id, status, expires_at	order_id (TEXT FK referencing orders.id), product_id (TEXT), variant_id (TEXT NULL), quantity (INTEGER), status (TEXT: 'ACTIVE', 'COMMITTED', 'RELEASED_EXPIRED', 'RELEASED_CANCELLED'), created_at (TEXT), expires_at (TEXT), resolved_at (TEXT NULL)	Tracks **Q6 atomic stock reservations**. Created at order placement with expires_at = created_at + 24 hours for unpaid orders. Prevents double-releasing stock.
payment_evidence	id (TEXT UUID PK) Index: order_id, verification_status	order_id (TEXT FK referencing orders.id), private_r2_key (TEXT UNIQUE), submitted_reference (TEXT), mime_type (TEXT), size_bytes (INTEGER), verification_status (TEXT: 'PENDING', 'VERIFIED', 'REJECTED'), verified_by_admin_id (TEXT FK NULL), verification_note (TEXT NULL), created_at (TEXT), verified_at (TEXT NULL)	Enforces **Section 4.2**: unpredictable private_r2_key never exposed publicly; tracks independent verification by one of the 3 admins before payment_status can become 'PAID'.
cancellation_requests	id (TEXT UUID PK) Index: order_id, status	order_id (TEXT FK referencing orders.id), customer_reason (TEXT), status (TEXT: 'PENDING_REVIEW',	Enforces **Q19 (Option A)**: customer request only logs 'PENDING_REVIEW' and sets order_status =

Table Name	Primary Key & Indexes	Key Columns & Data Types	Architectural Purpose & Invariant Rules
		'APPROVED', 'REJECTED'), reviewed_by_admin_id (TEXT FK NULL), admin_decision_note (TEXT NULL), requested_at (TEXT), reviewed_at (TEXT NULL)	'CANCELLATION_REQUESTED'; only an admin decision releases stock or initiates refund workflow.
refund_records	id (TEXT UUID PK) Index: order_id, refund_status	order_id (TEXT FK referencing orders.id), amount_pkr (INTEGER), refund_status (TEXT: 'APPROVED_PENDING_EXECUTION', 'PARTIALLY_REFUNDED', 'COMPLETED_VERIFIED'), approved_by_admin_id (TEXT FK), executed_by_admin_id (TEXT FK NULL), settlement_reference (TEXT NULL), created_at (TEXT), completed_at (TEXT NULL)	Enforces **Q7 & Q19**: separates *refund approval* ('APPROVED_PENDING_EXECUTION') from *verified return of funds* ('COMPLETED_VERIFIED' requiring external settlement reference).
customer_notifications	id (TEXT UUID PK) Index: order_id, created_at	order_id (TEXT FK referencing orders.id), channel (TEXT: 'WEBSITE', 'EMAIL'), event_type (TEXT), title (TEXT), message_body (TEXT), is_read_on_site (INTEGER DEFAULT 0), email_delivery_status (TEXT: 'NOT_ENABLED', 'QUEUED', 'SENT', 'FAILED'), created_at (TEXT)	Enforces **Confirmation #3**: persists guaranteed **website notifications** visible in the customer's Order Tracking view while tracking optional **email** delivery state separately.

Table Name	Primary Key & Indexes	Key Columns & Data Types	Architectural Purpose & Invariant Rules
store_settings & contact_submissions	key (TEXT PK) / id (TEXT UUID PK)	store_settings: key, value_json, updated_by_admin_id, updated_at. contact_submissions: id, name, email, phone, order_number, message, status, created_at.	store_settings backs **Q3** (admin-configurable payment methods, shipping fees, email notification toggle, security settings). contact_submissions backs **Q8** server-validated contact form storage.

Dual-Boundary R2 Storage and Client-Assisted Image Optimization

Cloudflare R2 provides persistent object storage with a free-tier allowance of 10 GB storage, 1,000,000 Class A (write/list) operations, and 10,000,000 Class B (read) operations per month with zero egress bandwidth fees. To satisfy both **Q15 (Product Image Uploads)** and **Section 4.2 (Private Payment Evidence)** without incurring paid image-transformation subscriptions, the architecture enforces strict logical and cryptographic isolation inside R2 across two segregated storage boundaries.

The **Public Catalog Media Boundary** (catalog/watches/{product_id}/{uuid}.webp) governs all storefront watch photography. Only authenticated administrators from the three authorized accounts may upload product images via POST /api/admin/products/:id/images. Prior to transmitting the file from the Android browser, the Admin Panel executes an HTML5 <canvas> pre-processor that strips camera EXIF metadata (such as embedded GPS coordinates), generates responsive dimensions (1200px for high-detail zoom views and 400px for catalog card thumbnails), and transcodes the image into compressed **WebP** format (< 300 KB per asset). Because client-supplied MIME types and file extensions can be trivially spoofed, the receiving Cloudflare Pages Function inspects the raw binary magic bytes of the uploaded buffer (RIFF....WEBP, FF D8 FF for JPEG, or 89 50 4E 47 for PNG), enforces a strict server-side payload ceiling of 2 MB per image, and writes the object to R2 using a server-generated UUID key. Whenever an administrator replaces or deletes a watch image, the backend atomically updates or removes the corresponding row in D1 product_images and deletes the obsolete R2 object key so orphaned assets never accumulate against the 10 GB free-tier ceiling.

Conversely, the **Private Payment Evidence Boundary** (private-evidence/{order_id}/{crypto_random_uuid}.{ext}) isolates customer-submitted payment transfer receipts for bank transfer, Raast, or mobile wallet orders. Objects under this prefix (or dedicated private R2 bucket binding) have **no public R2 custom domain and no unauthenticated public read route**. A customer may upload payment evidence (POST /api/orders/:order_number/payment-evidence) solely by presenting the valid Order Tracking Token for an order whose payment_status is 'PENDING' and whose order_status is not 'CANCELLED'. Upon verifying binary magic bytes and enforcing a 3 MB size limit, the server generates an unguessable 256-bit random object key (crypto.randomUUID()), writes the receipt to private R2 storage without ever overwriting prior submissions, and transitions payment_status to 'AWAITING_VERIFICATION' rather than 'PAID'. Private evidence can be inspected exclusively through the protected administrative endpoint GET /api/admin/orders/:id/payment-evidence/:evidence_id, which validates the administrator's active

session cookie on the server, streams the binary payload with Cache-Control: private, no-store, no-cache and Content-Security-Policy: default-src 'none'; img-src 'self', and records the inspection event in audit_logs.

Three-Admin Equal-Permission Security Model and Concurrency Control

In accordance with Confirmation #2 and Section 4.1 of the Handover, the administrative security architecture provides **three independent administrator accounts with identical, full permissions** across catalog management, order management, manual payment verification, refunds, account management, security settings, and audit-log inspection. Passwords are never stored in plaintext or weak digests, and credentials or API secrets are never embedded in frontend files or committed to GitLab. Utilizing the Web Crypto API natively available in the Cloudflare Pages Functions runtime (crypto.subtle), administrator passwords are salted with a unique 16-byte cryptographic random salt per account and hashed via **PBKDF2-HMAC-SHA256**, with verification performed using constant-time byte comparison (crypto.subtle.timingSafeEqual) to prevent timing side-channel leaks.
When any of the three administrators authenticates via POST /api/admin/auth/login, the server generates a 256-bit cryptographically random session token, stores only its SHA-256 digest in the D1 admin_sessions table with an 8-hour expiration window, and issues an HttpOnly; Secure; SameSite=Strict; Path=/api/admin cookie paired with a synchronizer **CSRF token** required in the X-CSRF-Token HTTP header on all mutating requests. Brute-force login abuse is mitigated at the database layer: every failed authentication attempt increments failed_login_attempts on the targeted admins row and writes a security alert containing the client IP (CF-Connecting-IP) to audit_logs. Five consecutive failed attempts within a 15-minute window automatically populate locked_until for 30 minutes, rejecting all login attempts during the cooldown window. Furthermore, while all three administrators share equal access to view the admins list, adjust security settings, verify payments, approve refunds, and inspect audit_logs, every sensitive administrative action requires **step-up password re-verification** (confirming the acting administrator's current password) and enforces the peer-protection invariant that at least two of the three administrator accounts must remain active (is_active = 1) at all times.

Four-Domain Decoupled Order State Machine and Atomic Stock Reservation

The order lifecycle strictly implements **Q6 (Atomic Stock Reservation with 24-Hour Expiry)**, **Q7 (Controlled Returns and Refunds)**, **Q19 (Customer Cancellation Requests)**, and **Q20 (Separate Four-Domain Status Tracking)**. Collapsing order progression into a single status column is prohibited because payment verification, physical package fulfillment, courier transit, and customer cancellation requests operate on asynchronous, independent timelines.
Because Cloudflare Pages Free Tier does not rely on paid external queue infrastructure, the **24-hour unpaid stock reservation expiry (Q6)** is enforced deterministically through **hybrid lazy-evaluation on read/write combined with an administrative batch sweep** inside Cloudflare D1. When a customer places an order (POST /api/orders), the backend executes a single atomic db.batch([...]) transaction in D1 that begins with a conditional stock reservation update: UPDATE product_variants SET reserved_stock = reserved_stock + ? WHERE id = ? AND is_active = 1 AND (total_stock - reserved_stock) >= ? (or directly on products for

single-SKU watches without variants). If the affected row count is zero, the batch aborts immediately, guaranteeing that concurrent checkout requests can never oversell the final unit of stock. Within that same atomic batch, the server inserts the orders record, immutable order_items price/product snapshots, a stock_reservations record with expires_at = datetime('now', '+24 hours'), and an initial on-site notification in customer_notifications. Subsequently, whenever a storefront product query, checkout validation, or administrative order dashboard request runs, a lightweight pre-transaction sweep evaluates any stock_reservations where status = 'ACTIVE' and expires_at < datetime('now') for orders whose payment_status remains 'PENDING'. Any expired reservation is atomically transitioned to 'RELEASED_EXPIRED', its reserved_stock quantity is decremented on the corresponding watch or variant, the associated order is transitioned to order_status = 'CANCELLED' and payment_status = 'FAILED', and both an audit log entry and a customer website notification are recorded.

The backend enforces strict state-machine invariants across the four separate status domains so inconsistent combinations can never occur:

Domain	Initial Status	Allowed Transitions & Guard Conditions	Cross-Domain Invariants & Inventory Rules
Order	Pending review	From Pending review to Confirmed, Cancellation requested, or Cancelled. From Confirmed to Cancellation requested or Cancelled. From Cancellation requested to Confirmed (if rejected) or Cancelled (if approved).	Customer cancellation request (Q19) **only** sets Cancellation requested and inserts a cancellation_requests row (PENDING_REVIEW). It **never** releases stock or cancels the order until one of the 3 admins approves it.
Payment	Pending	From Pending to Awaiting verification, Paid (COD upon delivery settlement), or Failed. From Awaiting verification to Paid or Failed. From Paid to Refund pending, then to Partially refunded or Refunded.	Uploading a screenshot sets Awaiting verification (Section 4.2). Marking Paid requires an admin's explicit verification action and note. Approving a refund sets Refund pending; marking Refunded or Partially refunded requires a verified settlement reference (Q7).
Fulfillment	Not started	Sequential progression from Not started to Processing, then to Packed, and finally to	Cannot advance to Dispatched if order_status is Cancelled or

Domain	Initial Status	Allowed Transitions & Guard Conditions	Cross-Domain Invariants & Inventory Rules
		Dispatched.	Cancellation requested, or if a prepaid order's payment_status is not Paid. When Dispatched is reached, stock_reservations.status becomes COMMITTED and both total_stock and reserved_stock are permanently decremented.
Shipping	Awaiting shipment	From Awaiting shipment to Shipped, then to In transit (where verified), and to Delivered or Delivery issue. From Delivery issue to In transit or Delivered.	Manual admin-managed shipping at launch (Q14). Moving to Shipped requires recording the courier name and tracking reference (where applicable). Moving to Delivered requires delivery verification.

API Contract Specification and Server-Side Validation Rules

All dynamic operations are routed through **Cloudflare Pages Functions** (/functions/api/...), enforcing server-side input validation, output encoding, parameterized SQL queries (preventing SQL injection), and strict HTTP status codes:

Endpoint Route & Method	Access Control	Purpose & Validation Rules
GET /api/catalog/products GET /api/catalog/products/:slug	Public Storefront	Returns only status = 'PUBLISHED' watches with active variants, whole-PKR regular/sale prices, available stock (total_stock - reserved_stock), and public R2 image URLs.
POST /api/orders	Public (Rate-Limited + Turnstile/CSRF protected)	Validates watch/variant IDs against live D1 prices and stock (ignoring frontend-supplied prices), atomically reserves stock (Q6), generates an Order Reference + Tracking Token, creates an on-site notification, and triggers non-blocking email

Endpoint Route & Method	Access Control	Purpose & Validation Rules
		dispatch if verified/enabled.
GET /api/orders/:order_number/track	Customer (Requires Tracking Token / Auth)	Returns sanitized customer-facing status across the 4 domains (Q20) and the customer's **website notifications** (customer_notifications). Strips all internal admin notes, private R2 keys, and audit metadata.
POST /api/orders/:order_number/payment-evidence	Customer (Requires Tracking Token)	Validates binary magic bytes (JPEG, PNG, WEBP, PDF), enforces size limit (<= 3 MB), writes to private R2 with random UUID key, sets payment_status = 'AWAITING_VERIFICATION'.
POST /api/orders/:order_number/cancel-request	Customer (Requires Tracking Token)	Implements **Q19**: logs customer cancellation reason in cancellation_requests and transitions order_status to Cancellation requested without releasing stock or triggering refunds.
POST /api/contact	Public (Rate-Limited + Spam Protected)	Implements **Q8**: validates name, email/phone, optional order reference, and message length; persists submission in D1 contact_submissions.
POST /api/admin/auth/login POST /api/admin/auth/logout GET /api/admin/auth/me	Admin Auth (Throttled)	Authenticates any of the **3 independent administrator accounts**, enforces 5-attempt lockout, issues HttpOnly; Secure; SameSite=Strict cookie + CSRF token, and writes login event to audit_logs.
GET / POST / PUT / DELETE /api/admin/products... POST /api/admin/products/:id/images	Protected Admin (All 3 Equal Admins)	Full CRUD for watches, categories, optional variants (Q5), stock adjustments, and R2 product image uploads/replacements (Q15) with state_version optimistic locking and audit logging.
GET / PATCH /api/admin/orders/:id/status POST /api/admin/orders/:id/verify-pay	Protected Admin (All 3 Equal Admins)	Executes validated transitions across the 4 status domains (Q20), streams private payment evidence (Section 4.2),

Endpoint Route & Method	Access Control	Purpose & Validation Rules
ment POST /api/admin/orders/:id/review-cancellation POST /api/admin/orders/:id/refunds		approves/rejects cancellation requests (Q19), and records refund approvals vs. verified refund completions (Q7) with state_version checks.
GET / PUT /api/admin/accounts GET / PUT /api/admin/settings GET /api/admin/audit-logs GET /api/admin/backups/export	Protected Admin (All 3 Equal Admins + Step-Up Password Check)	Implements **Confirmation #2**: allows all 3 equal-permission admins to manage admin accounts (with anti-lockout guard), configure payment/shipping/security settings (Q3), inspect append-only audit_logs, and export verified D1 JSON/SQL backups (Q13).

Repository Documentation Architecture

Once implementation is authorized in Phase 1, the GitLab repository will maintain four living Markdown documents as the durable source of truth across sessions. Specifically, **AI_CONTEXT.md** preserves the overarching project purpose, mobile-first development constraints, confirmed decisions (including the four authoritative overrides), current phase and gate status, active blockers, and safe continuation instructions for any future assistant. Alongside it, **BLUEPRINT.md** houses the complete technical architecture, the 15-table D1 relational schema, R2 dual-boundary storage rules, the three-admin equal-permission security model, the four-domain order state machine, API contracts, and phased acceptance criteria established in this report. Every meaningful code change, schema migration, and configuration update is chronologically recorded in **CHANGELOG.md**, while **TESTING.md** documents concrete test procedures, Android execution steps, actual observed outputs, known limitations, and verification evidence for each of the 18 launch blockers—strictly distinguishing between *planned*, *implemented*, *tested*, and *deployed* states.

Remaining Unresolved Items, External Verifications, and Launch Blockers

To respect the directive not to restart discovery from scratch while never inventing missing answers, all remaining open items are consolidated into three structured matrices covering **(1) Focused Material Decisions Required from the Project Owner**, **(2) Mandatory External and Technical Verifications**, and **(3) The 18 Launch Blockers**.

Focused Material Decisions Required from the Project Owner

Only five specific business and functional parameters remain unconfirmed from the accessible project record. None of these prevent initiating Phase 1 (Repository & Static Foundation) once START PROJECT is issued, or the project owner may confirm them directly alongside the START PROJECT authorization:

Unresolved Item ID	Decision Domain	Specific Question Requiring Confirmation	Recommended Default (If Approved by Owner)
MD-1 (Q1 & Q5 refinement)	**Watch Categories & Variant Labels**	What initial watch categories should appear in the storefront navigation (e.g., *Men's Watches, Women's Watches, Chronograph / Luxury, Minimalist / Casual*), and what default variant types are needed for the first 15 watches (e.g., *Strap Material/Color, Dial Color, Case Size*)?	Categories: *Men's, Women's, Luxury, Minimalist*. Default Variant Label: *Strap / Dial Finish* (optional per watch).
MD-2 (Q9–Q12, Q19, Q20)	**Customer Checkout & Order Tracking Access Mode**	Should launch checkout operate as **Guest Checkout with a cryptographic Order Reference + Tracking PIN/Link** (granting immediate access to 4-domain status, website notifications, payment receipt upload, and cancellation requests), or is mandatory/optional **Customer Account Registration** desired at launch?	**Guest Checkout + Secure Order Tracking Token/PIN** (zero mobile friction, full notification and cancellation support).
MD-3 (Q23 confirmation)	**Shipping Charge Policy**	Is the previously noted **PKR 250** delivery fee confirmed as a flat nationwide charge across Pakistan at launch, or do charges vary by region, COD vs. prepaid, or a free-shipping order threshold?	**PKR 250 flat fee** stored in store_settings (configurable at any time by any of the 3 admins via Shipping Settings).
MD-4 (Q7 & Q19 confirmation)	**Return, Exchange & Cancellation Terms**	What specific return/exchange window (e.g., 7 days from verified delivery for unworn watches in	Cancellation requests permitted before Shipped; return inspection window defined in

Unresolved Item ID	Decision Domain	Specific Question Requiring Confirmation	Recommended Default (If Approved by Owner)
		original packaging) and cancellation cut-off rule (e.g., cancellation requests allowed prior to Shipped status) should be published and enforced?	store_settings after legal/business verification.
MD-5 (Confirmation #2 & Q16)	**Three Administrator Email Identifiers**	What are the three distinct email identifiers to configure for the three equal-permission administrator accounts when Phase 7 is reached (e.g., admin1@zeecollection.tech, admin2@zeecollection.tech, admin3@zeecollection.tech, or specific named addresses)?	Three distinct @zeecollection.tech addresses confirmed prior to Phase 7 (passwords never shared in chat).

Mandatory External and Technical Verifications

In accordance with Part 5 of the Handover and Section 14 of the Master Prompt, the following external dependencies must be inspected and verified during the relevant phases before any dependent feature is enabled or claimed operational:

Verification Area	Specific Items to Inspect & Verify	Target Phase	Known Technical Constraint / Architectural Mitigation
1. Cloudflare Account, Domain & R2 Activation	Inspect whether zeecollection.tech is active, which registrar holds it, whether Cloudflare nameservers are active, and whether the Cloudflare account has **D1** and **R2** enabled.	**Phase 1 & Phase 2**	Provisioning Cloudflare R2 (even on the 10 GB/month Free Tier) requires a verified payment method on file in the Cloudflare account. Must be verified early before Phase 4 image uploads.
2. GitLab & SPCK Mobile Deployment Path	Verify whether an empty GitLab repository exists or needs creation, confirm PAT token cloning/pushing from	**Phase 1**	User must hold Maintainer/Owner on the GitLab repo and push at least one commit before binding the Production branch

Verification Area	Specific Items to Inspect & Verify	Target Phase	Known Technical Constraint / Architectural Mitigation
	SPCK Editor on Android, and verify native Cloudflare Pages to GitLab connection.		in Cloudflare Pages.
3. Zoho Mail (info@zeecollection.tech) & Outbound Email Path	Verify whether the Zoho Mail account is active, check Cloudflare DNS MX, SPF, and DKIM records, and test whether outbound transactional email can be dispatched from Cloudflare Pages Functions.	**Phase 6 & Phase 8**	Zoho Mail Free restricts external SMTP on standard free plans or caps SMTP at 50 emails/day (10/hour), and Cloudflare Functions use HTTP fetch(). **Website notifications** in D1 serve as the primary guaranteed notification channel; email remains toggled off until verified.
4. WhatsApp Click-to-Chat (+923408133043)	Verify that +923408133043 is active on WhatsApp and that https://wa.me/923408133043 opens properly with pre-filled support text on Android and iOS browsers.	**Phase 5 & Phase 6**	Used strictly for click-to-chat customer support, never assumed to be automated messaging or order persistence.
5. Dispatch Locality & Courier Operations	Confirm exact dispatch locality around **Kotla / Gujrat, Punjab**, and verify courier (e.g., Leopards or alternative) pickup availability, delivery coverage, COD settlement cycle, and fee structure.	**Before Launch (Phase 6 / 9)**	Launch uses manual admin-managed shipping (Q14); no automated courier API will be built until a provider is selected and verified.
6. Candidate Payment Methods	Verify merchant eligibility, account details, settlement workflow, and fee structures for candidate methods (COD, Bank Transfer, Raast, Easypaisa/JazzCash).	**Before Launch (Phase 6 / 9)**	Only verified payment methods may be enabled in Admin Payment Settings (Q22); all advance transfers require manual verification by one of the 3 admins

Verification Area	Specific Items to Inspect & Verify	Target Phase	Known Technical Constraint / Architectural Mitigation
			(Section 4.2).
7. Pakistani Legal & Financial Recordkeeping	Verify customer-facing privacy, return/refund disclosures, and statutory retention periods for financial/payment evidence records under applicable Pakistani regulations.	**Before Launch (Phase 8 / 9)**	Payment evidence deletion/retention schedules in R2 will not be automated until retention requirements are confirmed.

The 18 Launch Blockers and Acceptance Evidence Matrix

Per **Q21 (Option A)** and Part 6 of the Handover, the store cannot be declared launch-ready simply because pages render visually or a build succeeds; documented verification evidence must be recorded in TESTING.md across all 18 mandatory launch blockers:

Blocker ID	Launch Blocker Domain	Mandatory Verification Requirement & Evidence Standard	Resolution Phase
LB-01	**Persistent Catalog**	Product creation, edits, pricing, and variants must save and retrieve authoritatively from Cloudflare D1 (zero reliance on browser storage or static JSON).	Phase 3 & Phase 9
LB-02	**Persistent Image Storage**	Watch images must upload, optimize, persist, replace, and serve accurately through Cloudflare R2.	Phase 4 & Phase 9
LB-03	**Server-Side Authorization**	Every /api/admin/* endpoint must enforce server-side session and CSRF verification rather than relying on frontend UI hiding.	Phase 7 & Phase 8
LB-04	**Verified Three-Admin Authentication**	Independent logins for all 3 equal-permission administrators, PBKDF2 password hashing, session expiry/revocation, step-up	Phase 7 & Phase 9

Blocker ID	Launch Blocker Domain	Mandatory Verification Requirement & Evidence Standard	Resolution Phase
		re-authentication, and 5-attempt lockouts must be tested.	
LB-05	**Reliable Inventory Control**	Server-side price/stock validation, atomic D1 stock reservation at order placement, concurrency-safe updates, and 24-hour unpaid reservation expiry must prevent overselling.	Phase 6 & Phase 9
LB-06	**Verified Payment Activation**	Only payment methods that have been externally verified for merchant eligibility and operational readiness may be enabled in production settings.	Phase 6 & Phase 9
LB-07	**Independent Payment Verification**	No order may transition to Paid from a customer screenshot alone without an explicit, logged verification decision by one of the 3 administrators.	Phase 7 & Phase 9
LB-08	**Private Payment Evidence Security**	Payment receipts in R2 must use unpredictable UUID keys, block all public reads, prevent overwrites, validate binary magic bytes, and stream only to authenticated administrators.	Phase 4, Phase 6 & Phase 8
LB-09	**Confirmed Shipping Policy**	Delivery charges, dispatch procedures from Kotla/Gujrat, and coverage terms must be finalized and reflected accurately at checkout.	Phase 6 & Phase 9
LB-10	**Confirmed Returns &**	Customer-facing return,	Phase 8 & Phase 9

Blocker ID	Launch Blocker Domain	Mandatory Verification Requirement & Evidence Standard	Resolution Phase
	Refunds Terms	exchange, cancellation, and refund policies must be finalized, verified against Pakistani requirements, and published.	
LB-11	**Decoupled Four-Domain Status Workflow**	Order, Payment, Fulfillment, and Shipping statuses must operate independently with validated transitions, 3-admin optimistic concurrency locking, and append-only audit logs.	Phase 6, Phase 7 & Phase 9
LB-12	**Tested Backup & Recovery**	Documented and tested backup and restore procedures for critical D1 tables and R2 objects must be executed and verified within free-tier constraints.	Phase 9
LB-13	**Deployment & Custom Domain Verification**	zeecollection.tech DNS routing, HTTPS certificate validity, and Cloudflare Pages Functions execution must be verified on the live domain.	Phase 1 & Phase 9
LB-14	**Launch Acceptance Evidence**	Complete test procedures, edge-case checks, and actual observed results must be recorded in TESTING.md.	Phases 1–9
LB-15	**Continuity Documentation**	AI_CONTEXT.md, BLUEPRINT.md, CHANGELOG.md, and TESTING.md must accurately reflect the verified state of the repository.	Phases 1–9
LB-16	**Verified Mobile-First**	Every essential	Phase 1 & Phase 2

Blocker ID	Launch Blocker Domain	Mandatory Verification Requirement & Evidence Standard	Resolution Phase
	Workflow	development, database, and deployment operation must have a verified Android-compatible execution path (SPCK Editor + GitLab Web + Cloudflare Mobile Dashboard).	
LB-17	**Free-Tier Cost & Quota Verification**	D1, R2, and Pages Functions usage patterns must be verified against current Cloudflare Free Tier quotas and overage behaviors.	Phase 2 & Phase 9
LB-18	**External Channel Verification**	Website notifications, verified email (if enabled), contact form persistence, and WhatsApp click-to-chat (+923408133043) must be tested before being advertised as operational.	Phase 5, Phase 6 & Phase 9

Strictly Gated Phased Execution Roadmap

The implementation of ZEE COLLECTION is divided into nine strictly gated phases following this Phase 0 consolidation. In accordance with Part 9 of the Handover and Section 8 of the Master Prompt, no two phases may ever be combined into a single response, and work will come to a complete halt at the conclusion of each phase until the project owner reviews the verification evidence and issues the exact completion gate phrase.

Phase	Phase Title & Scope	Core Deliverables & Verification Focus	Required Authorization Gate to Enter Phase
Phase 0	**Discovery Reconciliation & Corrected Blueprint** *(Current Phase)*	Consolidated architecture blueprint, reconciled decision register (Watches catalog, 3 equal admins, Website + Email notifications & WhatsApp click-to-chat), D1 schema, R2	*Completed in this report — awaiting START PROJECT.*

Phase	Phase Title & Scope	Core Deliverables & Verification Focus	Required Authorization Gate to Enter Phase
		dual-boundary design, and unresolved item isolation.	
Phase 1	**Mobile-First Foundation, GitLab Repo, Baseline Storefront & Initial Pages Deployment**	Inspect current Cloudflare/domain/GitLab state on Android; configure GitLab repository + SPCK Editor PAT sync; create AI_CONTEXT.md, BLUEPRINT.md, CHANGELOG.md, TESTING.md, and luxury black/white/gold responsive HTML/CSS shell; verify GitLab-to-Cloudflare Pages deployment.	**START PROJECT**
Phase 2	**Cloudflare D1 Database Provisioning, Schema Migration & Backend Foundation**	Provision D1 database (zeecollection-db) via mobile dashboard; bind to Pages Functions; execute the 15-table SQL schema creation; build core server-side middleware (error handling, CORS, security headers, health check) and verify persistence.	**Phase 1 completely done**
Phase 3	**Watch Catalog API, Categories, Variants & Persistent D1 Integration**	Implement server-side public and admin watch catalog APIs (/api/catalog/*), category management, optional watch variants (Q5), whole-PKR price validation, and future-proof extensibility for rings and glasses.	**Phase 2 completely done**
Phase 4	**Cloudflare R2 Dual-Boundary Storage & Image Optimization Pipeline**	Verify R2 activation; configure public watch image storage vs. strictly private payment	**Phase 3 completely done**

Phase	Phase Title & Scope	Core Deliverables & Verification Focus	Required Authorization Gate to Enter Phase
		evidence storage; implement Android client-side HTML5 Canvas WebP compression + server-side magic-byte validation and safe image replacement/deletion (Q15).	
Phase 5	**Storefront Watch Catalog UI, Product Detail Galleries, Search & Click-to-Chat Support**	Connect storefront Home, Shop, Watch Detail, About, Contact (Q8), Shipping, and Returns pages to live D1/R2 APIs; implement search, category/price filtering, loading/empty/error states, and WhatsApp click-to-chat (+923408133043) support links.	**Phase 4 completely done**
Phase 6	**Shopping Cart, Checkout, 24h Atomic Stock Reservation, Order Tracking & Notifications**	Implement browser cart with server-side checkout price/stock validation, atomic D1 stock reservation + 24h expiry (Q6), 4-domain status tracking (Q20), private payment evidence upload (Section 4.2), customer cancellation request (Q19), **website notifications**, and modular email dispatcher (Confirmation #3).	**Phase 5 completely done**
Phase 7	**Three-Admin Equal-Permission Security System, Order/Refund Workflows & Audit Logs**	Implement the **3 independent administrator accounts** with equal permissions (Confirmation #2),	**Phase 6 completely done**

Phase	Phase Title & Scope	Core Deliverables & Verification Focus	Required Authorization Gate to Enter Phase
		PBKDF2 auth, session cookies, CSRF, brute-force lockouts, optimistic concurrency locking, payment verification UI, cancellation/refund management (Q7, Q19), account/security settings, and append-only audit_logs viewer.	
Phase 8	**Security Hardening, Accessibility, SEO, Mobile Performance & Policy Finalization**	Comprehensive security audit (CSP, rate limits, input sanitization, private R2 access denial tests), mobile performance tuning, SEO metadata/structured data for watches, and verified customer-facing policy pages.	**Phase 7 completely done**
Phase 9	**Backup/Recovery Verification, Custom Domain Cutover & 18-Blocker Launch Acceptance**	Execute and test D1/R2 backup and recovery procedures (Q13), verify zeecollection.tech DNS/HTTPS behavior, verify Zoho/email/payment/shipping settings, and complete all 18 launch blocker tests in TESTING.md (Q21).	**Phase 8 completely done** (Completed via **Phase 9 completely done**)

Pre-Implementation Hold and Authorization Gate

The corrected Phase 0 project blueprint, 15-table D1 data model, dual-boundary R2 storage strategy, three-administrator equal-permission security architecture, four-domain order state machine, and unresolved verification matrices are now consolidated. **Implementation remains paused, and no files, code, or infrastructure resources have been created or modified**. The project owner may now provide preferences on any of the five focused material decisions (MD-1 through MD-5) and, when ready to authorize the start of **Phase 1 only**, issue the exact authorization phrase:

START PROJECT

Works cited

1. Free Business Email: 5 Real Options Compared 2026 - meetergo, https://meetergo.com/en/magazine/free-business-email 2. I can't send any emails - DNS & Network - Cloudflare Community, https://community.cloudflare.com/t/i-cant-send-any-emails/662902 3. zoho smtp limit for free users - Zoho Cares, https://help.zoho.com/portal/en/community/topic/zoho-smtp-limit-for-free-users 4. Free Email Hosting for Custom Domain in 2026 - Truehost, https://truehost.com/free-email-hosting-for-custom-domain/ 5. How to Create Your Own Email Domain for Free In 2026, https://wpmailsmtp.com/how-to-create-email-domain-for-free/ 6. How to migrate your email to Zoho Mail for free | Cultura Digital, https://yagogonzalez.com/en/how-to-migrate-your-email-to-zoho-mail-for-free/ 7. Overview · Cloudflare Pages docs, https://developers.cloudflare.com/pages/ 8. Pricing · Cloudflare Workers docs, https://developers.cloudflare.com/workers/platform/pricing/ 9. Setting Up a Custom Email with Zoho Mail and Cloudflare, https://giftegwuenu.com/custom-email-zoho-cloudflare/ 10. Limits · Cloudflare D1 docs, https://developers.cloudflare.com/d1/platform/limits/ 11. Build and Host Full-Stack App with Database for Free - Startup, https://zerocoststartup.com/blog/build-and-host-full-stack-app-with-database-for-free 12. I was thrilled to use Cloudflare Pages+D1 until saw R2 need Credit, https://www.reddit.com/r/CloudFlare/comments/1s4bdml/i_was_thrilled_to_use_cloudflare_pagesd1_until/ 13. GitLab integration · Cloudflare Pages docs, https://developers.cloudflare.com/pages/configuration/git-integration/gitlab-integration/ 14. Spck Editor Getting Started, https://docs.spck.io/en/editor 15. Git integration guide · Cloudflare Pages docs, https://developers.cloudflare.com/pages/get-started/git-integration/ 16. Spck Editor / Git Client - Apps on Google Play, https://play.google.com/store/apps/details?id=io.spck 17. Creating an App Token - Spck Editor Docs - GitHub Pages, https://spckio.github.io/spck-documentation/create-app-token.html 18. APK Download for Android | Aptoide - Spck Editor / Git Client, https://spck-editor.en.aptoide.com/app 19. Is there any way to import a private GitLab repository from an, https://community.cloudflare.com/t/is-there-any-way-to-import-a-private-gitlab-repository-from-an-organization/846815 20. Cloudflare R2 Pricing 2026: Free Tier Limits, Egress Costs & When, https://nayankyada.com/blog/cloudflare-r2-pricing-2026-free-tier-limits-egress-costs-when-to-upgrade